LearnLens

Transform Any Topic Into Immersive Media — Hackathon Implementation Plan

Team Size: 4 Co-Founders

Format: Hackathon Sprint

Priority: Demo Polish > Feature Completeness

Date: March 2026

PROJECT OVERVIEW

Problem: People use ChatGPT to learn, but the output is always text — walls of paragraphs that are passive and forgettable.

Solution: LearnLens lets users type any topic and choose a rich media format as their output — an original image, a song, or an explanatory video — turning learning into an experience rather than a reading exercise.

Core Loop: User types topic → selects output format → AI generates immersive content in real time → user learns through seeing, hearing, or watching → output is saved and shareable via a persistent link.

OUTPUT MODALITIES

🖼️

Visual / Image

Topic rendered as a rich, informative visual — diagrams, scenes, or infographic-style art.

Imagen 3 / Ideogram

🎵

Song / Audio

Topic turned into a memorable original song with lyrics that teach the concept.

Google Lyria (MusicFX)

🎬

Video

Short explainer video generated from the topic with visuals and narration.

Veo 2 / Runway Gen-3

TECH STACK

Next.js 14 (Frontend) · TailwindCSS · FastAPI (Backend) · Python 3.11 · Google Lyria API · Imagen 3 / Ideogram API · Veo 2 / Runway API · Gemini (prompt synthesis) · WebSockets (real-time progress) · Cloudflare R2 / S3 (media storage) · Redis (job queue + caching) · Vercel · Railway · Cloudflare CDN

SYSTEM ARCHITECTURE

Client (Next.js)

- Input UI
- WebSocket listener
- Media players
- Share page (/s/[id])

→

API Server (FastAPI)

- POST /generate
- WebSocket /ws/[job]
- GET /output/[id]
- Gemini prompt builder

→

AI Layer

- Imagen 3
- Google Lyria
- Veo 2 / Runway
- Gemini (prompts)

→

Storage Layer

- Cloudflare R2 / S3
- Redis job queue
- Redis output cache
- CDN for media URLs

TEAM ROLES & IMPLEMENTATION PLANS

Person 1 — Frontend Engineer

Next.js · TailwindCSS · WebSocket client

Phase 1 — Core UI

- Scaffold Next.js 14 app with Tailwind
- Build landing page with hero + demo CTA
- Main input page: topic text field + output mode selector (Image / Song / Video)
- Example topic chips ("World War 2", "Black Holes", "Photosynthesis")

Phase 2 — Real-time & Results

- Connect to WebSocket endpoint; show live generation progress bar
- Animated step-by-step status ("Crafting prompt… Generating audio… Done!")
- Image result viewer with download button
- Audio player with synced lyrics display for song output
- Inline video player for video output

Phase 3 — Share & Polish

Person 2 — Backend Engineer

FastAPI · Python · Gemini · Job orchestration

Phase 1 — API Foundation

- Set up FastAPI project with CORS for frontend
- POST /generate → creates job ID, enqueues to Redis, returns job ID
- GET /output/[id] → returns completed output metadata + CDN URLs
- Worker process: pulls from Redis queue, runs generation, stores result

Phase 2 — Prompt Engineering

- Gemini call to expand raw topic into modality-specific structured prompts
- Image prompt: rich visual scene + educational framing
- Music prompt: style, mood, lyric brief — also generate full lyrics as text
- Video prompt: scene sequence + narration script

Phase 3 — Integration & Deploy

- Unified response schema across all AI modules

- Share page at /s/[id] — loads saved output by ID from backend
- "Copy link" button for shareable URLs
- Smooth page transitions & animations
- Responsive design; deploy to Vercel

- Error handling + graceful fallback messages
- Deploy worker + API to Railway
- Document all endpoints for frontend and infra engineer

🤖

Person 3 — AI Integrations

Lyria · Imagen · Veo · API wrappers

Phase 1 — Image Generation

- Integrate Imagen 3 via Vertex AI or Ideogram API
- Accept structured prompt, return image bytes → hand off to storage layer
- Test with 5+ educational topics, tune prompt style for clarity

Phase 2 — Music Generation

- Integrate Google Lyria (MusicFX) via Google AI Studio or Vertex AI
- Return audio file bytes → hand off to storage layer
- Emit progress events (via backend) as generation proceeds
- Test genre styles: educational jingle, lo-fi, epic orchestral

Phase 3 — Video Generation

- Integrate Veo 2 (Google) or Runway Gen-3 API
- Generate 5–10s educational clip; return video bytes → storage layer
- Fallback: stitch Imagen frames + TTS narration if video API unavailable
- Coordinate with infra on upload format (MP4, HLS)

🛠️

Person 4 — Infra & Real-time Engineer

WebSockets · Redis · R2/S3 · CDN · DevOps

Phase 1 — Real-time Layer

- Implement WebSocket server endpoint /ws/[job_id] in FastAPI
- Push named progress events to client: prompted → generating → uploading → done
- Set up Redis (Upstash free tier) for job queue and pub/sub between worker and WS server
- Reconnect logic on client drop; fallback to HTTP polling if WS unavailable

Phase 2 — Media Storage & CDN

- Set up Cloudflare R2 bucket (or AWS S3) for storing generated images, audio, video
- Write upload_media() helper used by all AI modules after generation
- Return signed CDN URLs with TTL; serve media via Cloudflare CDN for fast global delivery
- Store output metadata (topic, type, CDN URL, timestamp) in Redis with 24h TTL

Phase 3 — Caching, Share Links & DevOps

- Cache repeated topic+type requests: same topic hits CDN instead of re-generating
- Generate short share IDs (nanoid); expose GET /output/[id] for the share page
- Set up environment config (.env) and secrets management for all API keys
- Wire Vercel (frontend) + Railway (backend) deployments; confirm end-to-end in prod

HACKATHON TIMELINE

Hour 0–2 — Kickoff

Setup & Alignment

- Confirm API access: Lyria, Imagen, Veo/Runway, Gemini (apply for keys immediately)
- Infra: spin up Redis (Upstash), create R2/S3 bucket, provision Railway project
- Agree on request/response JSON schema and WebSocket event names across all 4 roles
- Frontend scaffolds Next.js; Backend scaffolds FastAPI with stub endpoints

Hour 2–6 — Core Build

Parallel Development Sprint

- Frontend: input page + loading state + result layouts (stub with mock data)
- Backend: /generate + worker process running locally; Gemini prompt expansion working
- AI: image generation end-to-end; music wrapper in progress
- Infra: WebSocket server live; Redis queue working; R2 upload helper ready for AI modules

Hour 6–10 — Integration

Connect All the Pieces

- Frontend connects real WebSocket; progress bar animates live
- All 3 AI outputs wired into backend, uploading to R2, returning CDN URLs
- Share links working end-to-end (/s/[id] loads saved result)
- Full end-to-end test on at least 3 topics across all 3 output types

Hour 10–14 — Polish & Hardening

Demo-Ready

- Frontend: animations, responsive fixes, deploy to Vercel
- Infra: deploy backend + worker to Railway, confirm prod WebSocket and CDN URLs work
- Cache warm-up: pre-generate 4–5 demo topic outputs so they return instantly on stage
- Full team rehearsal of 3-minute pitch + live demo run-through (x2)

Demo Day — Presentation

Show Time

- Lead with the WOW moment: generate a song live, show the real-time progress bar filling
- Show image output for a visual-friendly topic (e.g. "The Solar System")
- Copy and share the output link live — show it opens on a phone
- Close with the vision: "What if every student learned through their preferred medium?"

LIVE DEMO FLOW

⌨️ **User Types Topic** "World War 2"

→

🎛️ **Choose Format** Song / Image / Video

→

📡 **Live Progress** WebSocket updates

→

🎉 **Experience It** Watch / Listen / View

→

🔗 **Share It** Persistent CDN link